

SIMON 系列轻量级分组密码故障立方攻击

马云飞¹, 王 韬¹, 陈 浩¹, 张 帆², 楼潇轩², 许鲁珉², 杨文兵³

(1. 机械工程学院 信息工程系, 河北 石家庄 050003; 2. 浙江大学 信息与电子工程学院, 浙江 杭州 310027;

3. 九八〇四厂军代室, 云南 曲靖 655000)

摘 要: 针对 SIMON 密码按位与 & 运算特性以及现有立方攻击与故障攻击的不足, 给出一种故障立方攻击方法. 根据线性 and 二次多项式数量确定候选故障注入轮; 利用差分特征表确定故障注入的具体位置; 利用离线阶段求得的大量低次多项式, 恢复部分轮密钥, 并结合密钥猜测攻击恢复全轮密钥. 结果表明: 对 SIMON32/64 进行故障立方攻击, 需要平均注入故障 69 次, 计算复杂度为 $2^{47.91}$, 优于现有立方攻击; 相比于差分故障攻击, 采用故障立方攻击方法确定故障位置更有效, 故障模型更易实现, 且整个攻击过程具有自动化程度高的特点. 该方法可为核心运算次数较低的轻量级分组密码提供借鉴.

关键词: 轻量级分组密码; SIMON; 立方攻击; 故障攻击

中图分类号: TP 309

文献标志码: A

文章编号: 1008-973X(2017)09-1770-10

Fault-cube attack on SIMON family of lightweight block ciphers

MA Yun-fei¹, WANG Tao¹, CHEN Hao¹, ZHANG Fan², LOU Xiao-xuan²,
XU Lu-min², YANG Wen-bing³

(1. Department of Information Engineering, Ordnance Engineering College, Shijiazhuang 050003, China;

2. College of Information Science and Electrical Engineering, Zhejiang University, Hangzhou 310027, China;

3. The Nine Eight Zero Four Military Representative Office, Qujing 655000, China)

Abstract: A fault-cube method was given aiming at the special property of And operation (&) in SIMON and the problem in previous cube attack and fault attack. The round-candidates for fault injection were identified according to the number of linear and quadratic equations. Positions for fault injection were determined by using a difference-characteristics table. Some round-keys were recovered by extracting low-degree equations during the off-line phase. Then, the entire round-keys were obtained with combination of guess-and-determine attack. The experimental results show that the attack on SIMON32/64 needs 69 fault injections on average and requires a compute complexity of $2^{47.91}$, which is better than the previous cube attack. Compared to differential fault attack, the fault-cube method is more effective in determining fault positions. Moreover, using the fault model is easier to realize and the attack process is of high automation. The fault-cube method will provide some ideas on other lightweight block ciphers with low-degree core operations as well.

Key words: lightweight block cipher; SIMON; cube attack; fault attack

SIMON 系列轻量级分组密码^[1]由美国国家安全局在 2013 年提出, 主要用于资源受限环境. SIMON

密码由于设计简单(大部分 SIMON 密码门电路数量小于 1 000)且灵活性较高, 在硬件和软件方面均

收稿日期: 2016-12-01.

网址: www.zjujournals.com/eng/fileup/HTML/201709011.htm

基金项目: 国家自然科学基金资助项目(61272491, 61309021, 61472357); 保密通信重点实验室基金资助项目(9140C110602150C11053).

作者简介: 马云飞(1992—), 男, 硕士生, 从事旁路立方攻击研究. orcid.org/0000-0002-2528-309X. E-mail: fcz1992@sina.com

通信联系人: 王韬, 男, 教授. orcid.org/0000-0002-9327-6019. E-mail: T_Wang_mail@163.com

表现出优越的性能,受到了广泛关注. SIMON 密码“&”运算的低次性使其易遭受各类攻击,因此针对 SIMON 密码的安全性评估研究较多,主要有线性攻击^[2-3]、不可能差分攻击^[4]、差分攻击^[5-6]以及代数攻击^[7]等.

目前,针对 SIMON 密码的立方攻击和故障攻击研究如下. Rabbaninejad 等^[8]对 SIMON32/64 进行立方和动态立方攻击,破解全部密钥的计算复杂度分别为 $2^{51.49}$ 和 $2^{51.09}$. Tupsamudre 等^[9]对 SIMON 系列密码进行了传统差分故障分析. 基于单比特故障模型的研究结论如下:“SIMON 系列密码倒数第 2 轮左半部分每注入 1 bit 故障可恢复最后一轮 2 bit 轮密钥”. 在文献[8]中,由于多项式次数和项数随轮数呈指数级增长,导致得到的立方体维数较大,计算复杂度较高,仅能实现约简轮 SIMON32/64 的攻击. 文献[9]利用差分推导进行故障定位,仅适用于倒数第 2 轮左半部分故障注入,单独对左半部分进行故障注入在实际攻击中较难实现. 此外,差分故障分析需要结合具体密码进行推导,自动化程度较差.

针对上述不足,本文将立方与故障结合,给出故障立方攻击方法. 目前旁路立方攻击采用的泄露模型主要是基于中间状态泄露的模型,如:单比特模型^[10-11]和汉明重模型^[12-13],而基于故障信息泄露的旁路立方攻击研究较少. 曾文^[14]针对流密码提出一种“fault-cube”方法,通过故障将高维立方体截成 2 个低维立方体,并对其分别进行立方攻击. 该方法对 1 056 拍以下的 Trivium 算法有效,计算复杂度不超过 2^{53} . Abdul-latif 等^[15]结合立方,对 KATAN 系列分组密码进行故障攻击,通过 115 次故障注入,将 KATAN32 的计算复杂度降低为 2^{59} .

本文利用故障立方攻击对 SIMON 系列密码进行安全性评估研究. 给出故障立方攻击基本原理和模型,在此基础上对 3 种 SIMON 系列密码进行攻击. 通过仿真实验验证了故障立方攻击有效性,并将该方法与立方攻击和故障攻击对比分析.

1 相关知识

1.1 SIMON 系列密码

SIMON 系列轻量级分组密码采用平衡 Feistel 结构,为了提高算法灵活性,设计者提供了 10 个版本,均可用 SIMON $2n/(en)$ 表示. 其中 n 为字长, $2n$ 为分组长度, en 表示密钥长度. 本文选取的研究对象是 SIMON32/64、SIMON48/72 和 SIMON64/96,对

应的算法轮数分别为 32、36、42 轮.

SIMON 系列密码的轮函数主要包含 3 种操作:按位异或($\oplus$)、按位与($\&$)、循环左移($\lll$). 设 L_r 和 R_r 分别表示经过 r 轮加密后中间状态的左半部分和右半部分, K_r 为轮密钥,则 SIMON 系列密码每一轮加密过程表示为

$$L_{r+1} = F(L_r) \oplus R_r \oplus K_r, R_{r+1} = L_r. \quad (1)$$

式中: $F(L_r) = ((L_r \lll 1) \& (L_r \lll 8)) \oplus (L_r \lll 2)$.

SIMON 系列密码轮函数的具体结构如图 1 所示. SIMON 的核心为“&”运算,相比于代数方程最高次数为 3 的 S 盒、代数方程最高次数为 d 的模加运算(d 为运算单位二进制长度),“&”运算后的代数方程最高次数仅为 2,具有低次性的特点.

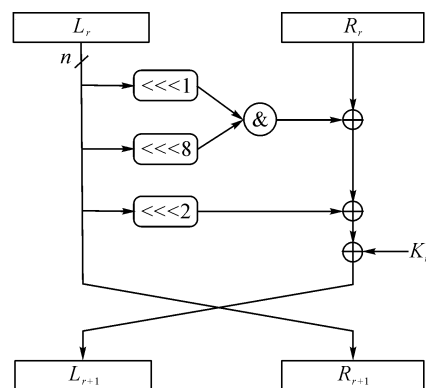


图 1 SIMON 系列分组密码轮函数

Fig. 1 Round function of SIMON family block ciphers

1.2 立方攻击

若将任意密码算法看作一个黑盒,则输出结果的每一位都能表示成关于 u 位明文变量 $v_1, v_2, \dots, v_u$ 以及 l 位密钥变量 $w_1, w_2, \dots, w_l$ 的多项式 $p(v, w)$. 假设 I 是明文集合 $\{v_1, v_2, \dots, v_u\}$ 的子集, I 中变量下标的集合为 $\{y_1, y_2, \dots, y_m\}$, 令

$$t_I = \prod_{y \in \{y_1, y_2, \dots, y_m\}} v_y,$$

则通过 t_I 可以将多项式 $p(v, w)$ 分解为

$$p(v, w) = t_I \cdot p_{S(I)} + q(v, w). \quad (2)$$

式中: $p_{S(I)}$ 为关于密钥 w 的多项式, $q(v, w)$ 为所有不能被 t_I 整除项的累积和. 特别地, 当 $p_{S(I)}$ 最高次等于 1 时, 称 t_I 为极大项, $p_{S(I)}$ 为超级多项式.

立方攻击实现的关键在于下列定理的成立.

定理 1^[16] 集合 $C_I = \{(v_1, v_2, \dots, v_u), \text{ 当 } y = y_1, y_2, \dots, y_m \text{ 时, } v_y = 0, 1; \text{ 当 } y \neq y_1, y_2, \dots, y_m \text{ 时, } v_y \text{ 的取值固定, 那么}$

$$\sum_{v_1, v_2, \dots, v_u \in C_I} p(v, w) \equiv p_{S(I)} \pmod{2}.$$

立方攻击可分为预处理阶段和在线分析阶段. 预处理阶段主要利用下式判断立方体对应的 $p_{S(D)}$ 是否为关于密钥的线性或二次函数, 其中 $a, b, c \in \{0, 1\}^l$, l 为密钥比特位数:

$$p_{S(D)}(0) + p_{S(D)}(a) + p_{S(D)}(b) = p_{S(D)}(a+b), \quad (3)$$

$$p_{S(D)}(0) + p_{S(D)}(a) + p_{S(D)}(b) + p_{S(D)}(c) + p_{S(D)}(a+b) + p_{S(D)}(a+c) + p_{S(D)}(b+c) = p_{S(D)}(a+b+c). \quad (4)$$

在线阶段利用预处理阶段获得的立方体, 通过 0/1 赋值, 对目标比特进行累加以获取对应多项式的值. 立方攻击的详细内容可参考文献[17].

1.3 故障攻击

故障攻击^[18]是一种利用密码设备在外界干扰下(温度、电压等)产生的故障行为或错误信息来恢复密钥的攻击方法. 1997 年, Biham 等^[19]结合差分分析与故障攻击, 提出了差分故障攻击.

差分故障攻击通常根据某一非线性部件的差分特性进行分析. 现有的故障注入技术按照成本和精度可分为高成本和低成本故障注入技术^[20]. 如果攻击者可以承受较高的预算, 则不能忽略一些高成本故障注入技术可能造成的潜在威胁. 高成本故障技术通过直接访问硅片, 可以将故障精确地注入芯片中, 攻击威胁较大. 利用这类技术可实现本文特定轮单比特故障注入模型, 该类技术主要有 3 种, 如表 1 所示.

表 1 高成本故障注入技术

Tab. 1 High-cost fault injection technology

技术	空间准确度	时间准确度	技术能力	永久性改动
光脉冲	中等	中等	中等	可能
激光束	高	高	高	可能
聚焦离子束	全部	全部	很高	有

2 故障立方攻击

本文故障模型设定如下:

a) 攻击者掌握了密码设备, 可在任意加密轮注入单比特随机翻转故障(即某一中间状态位由“0”变为“1”或由“1”变为“0”);

b) 攻击者只能控制故障注入轮数, 而不知道故障注入该轮的准确位置;

c) 攻击者能够重启密码设备, 使其恢复初始状态, 即密码设备允许反复注入单比特故障.

定义 1 设某 SIMON 密码算法 $\mathbf{A}$ 的加密轮数为 T , 明密文长度都是 $2n$, 轮密钥长度为 n . 攻击者在 t 时刻向 $\mathbf{A}$ 第 $s(1 \leq s \leq T)$ 加密轮中间状态 X 注入单比特随机故障 α , 生成错误密文 C' . 假设故障注

入了一个未知的位置 x_{s+i} (i 为该轮的某一比特位), 错误密文 C' 与正确密文 C 的差值是 $\Delta C = (\Delta c_0, \Delta c_1, \dots, \Delta c_{2n-1})$. 第 s 轮轮密钥表示为 $k_{(s-1)+0}, k_{(s-1)+1}, \dots, k_{(s-1)+n-1}$.

根据代数攻击思想, 在正常加密情况下密文 C 的某一比特 c_j 可以看成是关于某一轮 s 中间状态 $x_{s+0}, x_{s+1}, \dots, x_{s+2n-1}$ 和 $(s+1) \sim T$ 轮所有轮密钥 $k_{s+0}, k_{s+1}, \dots, k_{(T-1)+n-1}$ 的布尔函数. 将密文比特 c_j 看作立方攻击中的目标多项式 $p(v, kk)$, 将故障注入位 x_{s+i} 看作 t_I , 可以得到推论 1.

推论 1 假设 c_j 是正确密文 C 的某一比特, x_{s+i} 是 s 轮某一中间状态比特, 则在 x_{s+i} 处注入故障后, Δc_j 可表示为关于 s 轮中间状态 $x_{s+0}, x_{s+1}, \dots, x_{s+2n-1}$ 和 $(s+1) \sim T$ 轮所有轮密钥 $k_{s+0}, k_{s+1}, \dots, k_{(T-1)+n-1}$, 但不包含 x_{s+i} 的一个特定布尔函数.

证明: 如果将第 s 轮之后的加密过程分割出来单独考虑, 则 c_j 可以表示为关于 s 轮中间状态和 $(s+1) \sim T$ 轮所有轮密钥的布尔函数 $c_j(x_{s+0}, x_{s+1}, \dots, x_{s+2n-1}, k_{s+0}, k_{s+1}, \dots, k_{(T-1)+n-1})$, 令

$$X = \{x_{s+0}, x_{s+1}, \dots, x_{s+i-1}, x_{s+i+1}, x_{s+i+2}, \dots, x_{s+2n-1}\},$$

$$K = \{k_{s+0}, k_{s+1}, \dots, k_{(T-1)+n-1}\},$$

则有

$$c_j(x_{s+0}, x_{s+1}, \dots, x_{s+2n-1}, k_{s+0}, k_{s+1}, \dots, k_{(T-1)+n-1}) = x_{s+i} \cdot p(X, K) + q(X, K). \quad (5)$$

式中: $p(X, K)$ 和 $q(X, K)$ 表示关于集合 X 和集合 K 中元素的布尔函数.

将 x_{s+i} 看作式(2)的 t_I , 将集合 X 和集合 K 中变量看作未知量. 令集合 $C_I = \{0, 1\}$, 由于 x_{s+i} 和 $x_{s+i} \oplus 1$ 一定能取遍 C_I , 根据定理 1 可知

$$c_j(x_{s+0}, x_{s+1}, \dots, x_{s+i}, \dots, x_{s+2n-1}, K) \oplus c_j(x_{s+0}, x_{s+1}, \dots, x_{s+i} \oplus 1, \dots, x_{s+2n-1}, K) = \sum_{x_{s+i} \in C_I} c_j \equiv p(X, K) \pmod{2}. \quad (6)$$

故障注入后有

$$\Delta c_j = c_j(x_{s+0}, x_{s+1}, \dots, x_{s+i}, \dots, x_{s+2n-1}, K) \oplus c_j(x_{s+0}, x_{s+1}, \dots, x_{s+i} \oplus 1, \dots, x_{s+2n-1}, K). \quad (7)$$

综合式(6)、式(7)易得

$$\Delta c_j \equiv p(X, K) \pmod{2}. \quad (8)$$

综上, Δc_j 是一个不包含 x_{s+i} 的特定布尔函数. 利用推论 1 可将故障与立方结合, 高效地获取密钥, 故障立方攻击模型如图 2 所示. 由于现有立方攻击仅能提取线性和二次多项式, 故障立方攻击实现条件如下: 目标轮存在 x_{s+i} , 使得密文某比特 c_j 对应的 $p(X, K)$ 最高次 ≤ 2 . 通过初步研究发现, SIMON 密码低次性特点恰好符合这一要求.

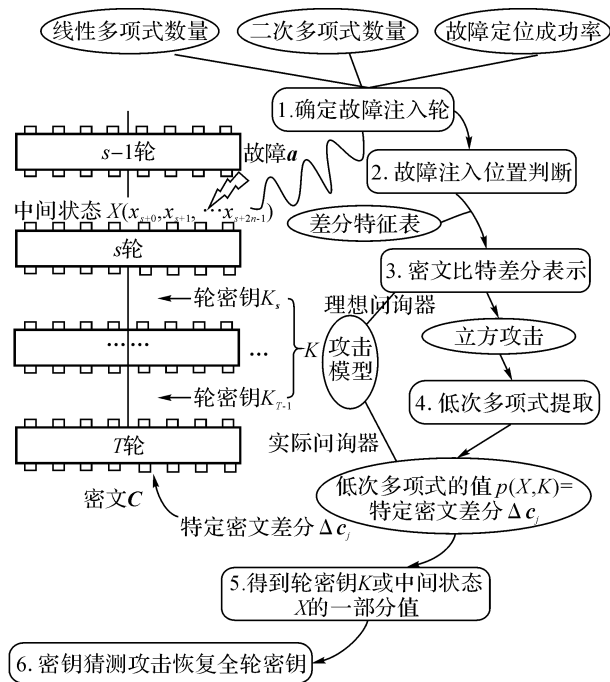


图2 故障立方攻击模型

Fig. 2 Model of fault-cube attack

3 对 SIMON32/64、SIMON48/72 和 SIMON64/96 的故障立方攻击

3.1 确定候选故障注入轮

为获得尽可能多的多项式以求解密钥,攻击者需要统计每一轮 $p(X, K)$ 为线性或二次多项式的数量,最多的即为候选故障注入轮. 假设要分析第 s 轮,令 x_{s+i} ($0 \leq i \leq 2n-1$) 表示第 s 轮中间状态比特, c_j ($0 \leq j \leq 2n-1$) 表示密文比特,循环遍历每一对 (x_{s+i}, c_j) ,考察其对应 $p(X, K)$,利用式(3)和(4)可判断该多项式是否为线性或二次多项式. 利用上述方法统计 3 种 SIMON 密码各轮线性及二次多项式数量,如图 3 所示, δ 为多项式个数, s 为故障注入轮. 观察发现: SIMON32/64、SIMON48/72、SIMON64/96 候选故障注入轮分别为 30 轮、34 轮和 40 轮.

3.2 确定故障注入具体位置

在确定了故障注入轮后,攻击者对该轮进行单比特随机故障注入. 为确定故障注入位置,每次选择一个 x_{s+i} ,观察密文比特的差分特征. 由式(8)可知,密文比特差分就是 $p(X, K)$,如果该多项式的值是常数“0”或“1”,则当故障注入 x_{s+i} 时,密文比特差分一定是特定值“0”或“1”,由此可构建某一轮 x_{s+i} 的密文差分特征表.

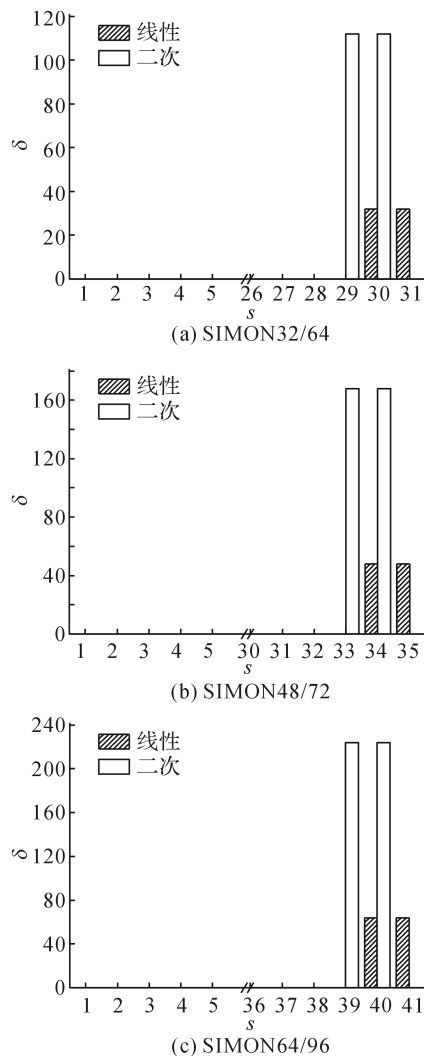


图3 各轮低次多项式数量统计

Fig. 3 Statistics of low-degree polynomials for different rounds

以 SIMON32/64 密码为例,计算第 30 轮差分特征表,结果如表 2 所示. 第 i 行代表在 x_{s+i} 处注入故障后得到的密文差分,第 j 列代表故障注入后第 j 比特的密文差分 Δc_j . “0”和“1”表示该位密文差分是确定值,“*”表示该位密文差分不确定. 如果 32 位密文差分值和表 2 中的第 i 行吻合,则证明故障注入位是 x_{s+i} .

利用差分特征表进行推断也存在一些不确定性. 例如,第 6 行和第 20 行分别为

$$001 * * 0 * 00000 * * 0000001 * 00000000 * 0, \\ 001 * 00000000 * 00000001 100000000000.$$

在某些情况下,密文差分可能同时满足第 6 和第 20 行. 为此,本文提出一种策略:如果存在密文差分同时满足某 2 行的情况,判定“*”较少的行对应的中间比特为故障注入位,证明如下.

表 2 SIMON32/64 第 30 轮故障注入差分特征表

Fig. 2 Difference-characteristics table for SIMON32/64 with 30th faulty round

$i \backslash j$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
0	*	0	0	0	0	0	*	*	0	0	0	0	1	*	*	0	0	0	0	0	0	0	0	0	*	0	0	0	0	1	*		
1	0	*	0	0	0	0	0	*	*	0	0	0	0	1	*	*	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	1	
2	*	0	*	0	0	0	0	0	*	*	0	0	0	0	1	*	1	*	0	0	0	0	0	0	0	0	0	*	0	0	0	0	0
3	*	*	0	*	0	0	0	0	0	*	*	0	0	0	0	1	0	1	*	0	0	0	0	0	0	0	0	0	*	0	0	0	0
4	1	*	*	0	*	0	0	0	0	0	*	*	0	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	*	0	0	0
5	0	1	*	*	0	*	0	0	0	0	0	*	*	0	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	*	0	0
6	0	0	1	*	*	0	*	0	0	0	0	0	*	*	0	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	*	0
7	0	0	0	1	*	*	0	*	0	0	0	0	0	*	*	0	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	*
8	0	0	0	0	1	*	*	0	*	0	0	0	0	0	*	*	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0
9	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	*	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0
10	*	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0
11	0	*	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0
12	0	0	*	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0
13	0	0	0	*	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0
14	0	0	0	0	*	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0
15	0	0	0	0	0	*	*	0	0	0	0	1	*	*	0	*	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0
16	0	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	1	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	0	1	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
20	0	0	1	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
21	0	0	0	1	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
22	0	0	0	0	1	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
23	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	*	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
24	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
25	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
26	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
27	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
28	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
29	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
30	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
31	0	0	0	0	0	0	0	*	0	0	0	0	0	1	*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

规则: 存在第 φ_1 行和第 φ_2 行同时满足密文差分, 这 2 行中“*”数量分别为 λ 和 μ . 如果 $\lambda \geq \mu$, 则输出 φ_2 , 否则输出 φ_1 .

证明: 如果将密文差分的每一比特看作相互独立的变量, 可以得出差分表第 φ_1 行和第 φ_2 行满足密文差分的概率分别为

$$P_\lambda = \left(\frac{1}{2}\right)^\lambda \left[1 - \left(\frac{1}{2}\right)^\mu\right], P_\mu = \left(\frac{1}{2}\right)^\mu \left[1 - \left(\frac{1}{2}\right)^\lambda\right].$$

若存在 $\lambda \geq \mu$, 则有

$$P_\lambda = \left(\frac{1}{2}\right)^\lambda \cdot \left[1 - \left(\frac{1}{2}\right)^\mu\right] \leq \frac{1}{2} \left[\left(\frac{1}{2}\right)^{2\lambda} + \left(1 - \left(\frac{1}{2}\right)^\mu\right)^2\right] = \frac{1}{2} - \left(\frac{1}{2}\right)^\mu + \left(\frac{1}{2}\right)^{2\mu+1} +$$

$$\left(\frac{1}{2}\right)^{2\lambda+1} \leq \frac{1}{2} - \left(\frac{1}{2}\right)^\mu + 2 \times \left(\frac{1}{2}\right)^{2\mu+1} = \frac{1}{2} - \left(\frac{1}{2}\right)^\mu + \left(\frac{1}{2}\right)^{2\mu} < \frac{1}{2}.$$

因此, 第 φ_2 行满足密文差分概率更大. $\lambda < \mu$ 的证明与此类似.

针对 SIMON32/64 第 30 轮差分特征表进一步研究发现, 所有可能出现冲突的行号组合 (6, 20)、(1, 31)、(0, 30)、(13, 27)、(4, 18)、(7, 21)、(9, 23)、(5, 19)、(15, 29)、(12, 26)、(2, 16)、(3, 17)、(11, 25)、(8, 22)、(14, 28)、(10, 24), 都有 $\lambda = 7$ 和 $\mu = 2$. 因此对于 SIMON32/64 第 30 轮差分特征表来说, “*”较多的行为故障注入位的概率:

$$P_\lambda = \left(\frac{1}{2}\right)^\lambda \left[1 - \left(\frac{1}{2}\right)^\mu\right] = \frac{3}{512}.$$

这一概率较小,后续仿真实验进一步证实了利用上述策略对 SIMON32/64 第 30 轮故障定位的成功率达到 100%. 实际成功率高于理论值,因为在本文的理论分析中,“*”是相互独立的,事实上可能并非如此. 以 SIMON32/64 第 30 轮第 0 bit 注入单比特故障为例说明“*”之间并非相互独立的.

如图 4 所示为手工推导故障传播路径,阴影部分为受影响比特位(比特值不一定发生变化),其余为不受影响的比特位. 另取表 2 第 0 行列于图 4 下方,假设 s 轮的比特值依次为 $L_{s,0}, L_{s,1}, \dots, L_{s,15}, R_{s,0}, R_{s,1}, \dots, R_{s,15}$, 而 $L(R)_{s,\eta}$ 受影响后表示为 $L(R)'_{s,\eta}$. 在差分表第 0 行的 7 个“*”分别为 $\Delta L_{T,0}, \Delta L_{T,6}, \Delta L_{T,7}, \Delta L_{T,13}, \Delta L_{T,14}, \Delta R_{T,8}, \Delta R_{T,15}$. 依次推导出密文每一个阴影部分的差分值:

$$\begin{aligned} \Delta L_{T,0} &= (L_{T-1,8} \& L_{T-1,1}) \oplus (L'_{T-1,8} \& L_{T-1,1}) \oplus 1, \\ \Delta L_{T,6} &= (L_{T-1,7} \& L_{T-1,14}) \oplus (L_{T-1,7} \& L'_{T-1,14}) \oplus (L_{T-1,8} \& L'_{T-1,8}), \\ \Delta L_{T,7} &= (L_{T-1,8} \& L_{T-1,15}) \oplus (L'_{T-1,8} \& L'_{T-1,15}), \\ \Delta L_{T,12} &= L_{T-1,14} \oplus L'_{T-1,14} = \Delta L_{T-1,14} = 1, \\ \Delta L_{T,13} &= (L_{T-1,14} \& L_{T-1,5}) \oplus (L'_{T-1,14} \& L_{T-1,5}) \oplus (L_{T-1,15} \& L'_{T-1,15}), \\ \Delta L_{T,14} &= (L_{T-1,15} \& L_{T-1,6}) \oplus (L'_{T-1,15} \& L_{T-1,6}), \\ \Delta R_{T,8} &= \Delta L_{T-1,8} = L_{T-1,8} \oplus L'_{T-1,8}, \\ \Delta R_{T,14} &= \Delta L_{T-1,14} = 1, \\ \Delta R_{T,15} &= \Delta L_{T-1,15} = L_{T-1,15} \oplus L'_{T-1,15}. \end{aligned}$$

以 $\Delta L_{T,0}$ 和 $\Delta R_{T,8}$ 为例,发现 $L_{T-1,8}$ 和 $L'_{T-1,8}$ 会同时对两者的值产生影响. 为进一步证明,随机选取 100 000 组明文、密钥及故障作为样本,计算得相关系数:

$$\rho = \frac{\text{Cov}(\Delta L_{T,0}, \Delta R_{T,8})}{\sqrt{D(\Delta L_{T,0})} \sqrt{D(\Delta R_{T,8})}} = -0.579\ 602,$$

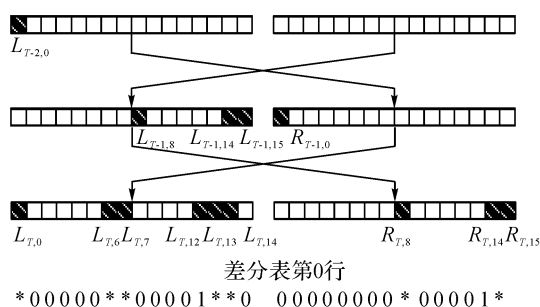


图 4 SIMON32/64 第 30 轮第 0 bit 故障传播路径

Fig. 4 Fault propagation path of 0 bit in SIMON32/64 with 30th faulty round

从而证明其并非相互独立. 类似地,可计算出其余“*”值之间也并非相互独立.

以随机明文 $P=0x\ 6120\ 6e69$, 初始密钥 $(k_3, k_2, k_1, k_0)=0x\ 1211\ 100a\ 0908\ 0201$ 为例,在第 30 轮注入单比特随机故障,如表 3 所示为利用差分特征表推断故障比特的过程. 分析结果表明:利用上述方法,可准确定位 0~31 故障比特.

3.3 求解轮密钥

利用故障注入将目标轮到最后一轮加密过程分离出来,等效于约简轮立方攻击. 求解轮密钥分为离线阶段和在线阶段. 离线阶段利用立方攻击,寻找低次多项式(线性或者二次). 若故障注入位 x_{s+i} 和密文比特 c_j 对应的 $p(X, K)$ 是低次多项式,则将 (x_{s+i}, c_j) 放入一个特定集合 Φ ,并记录该多项式. 在线阶段随机注入故障,如果判断故障注入位与集合 Φ 中某元素的 x_{s+i} 一致,则相应密文差分 Δc_j 即为对应多项式的值.

针对 SIMON32/64 第 30 轮的故障立方攻击找到 112 个二次多项式和 32 个线性多项式,表 4 列举了其中一部分. 在上述 144 个多项式中,排除冗余以及次数过高无法利用的多项式,筛选得到 32 个多项式,如表 5 所示.

对 SIMON48/72 第 34 轮实施故障立方攻击,筛选后得到 24 个二次多项式和 24 个线性多项式. 类似地, SIMON64/96 第 40 轮筛选后得到 32 个二次和 32 个线性多项式,受篇幅限制不再一一列举.

4 仿真实验与复杂度分析

4.1 利用差分特征表进行故障定位的成功率分析

编程模拟利用 SIMON32/64 各轮差分特征表(以下简称“差分表”)确定故障位置的过程. 定义故障注入轮到密文输出之间的故障传播轮数为故障深度,记为 $f_d = T - s$. 不同故障深度分别随机选取 100 组明文、密钥及故障注入位置,观察每组数据的密文差分在差分表中匹配的行数,如图 5 所示. 图中, ϵ 为实验样本, θ 为密文差分在差分表中匹配的行数. 分析实验结果发现:随故障深度 f_d 增加,密文差分在差分表中匹配的行数越来越多,由此也导致定位成功率下降.

进一步模拟 SIMON48/72 和 SIMON64/96 利用差分表定位的过程. 当故障深度 $f_d \in [1, 6]$ 时,这 3 种 SIMON 密码差分表定位情况如图 6 所示. 其中, χ 为故障定位成功率. 观察发现:1)差分表定位成功率随故障深度增加而呈下降趋势;2)这 3 种 SIMON 密码候选故障注入轮的差分表定位成功率

表 4 SIMON32/64 第 30 轮多项式举例

Tab. 4 Equations for example of SIMON32/64 in round 30

故障注入位	密文差分	多项式
x_{s+0}	Δc_0	$k_{s+1}x_{s+9} + x_{s+2}x_{s+9} + x_{s+3}x_{s+9} + x_{s+9}x_{s+17} + 1$
x_{s+2}	Δc_0	$k_{s+8}x_{s+9} + x_{s+0}x_{s+9} + x_{s+9}x_{s+10} + x_{s+9}x_{s+24}$
x_{s+3}	Δc_0	$k_{s+8} + x_{s+24} + x_{s+0}x_{s+9}$
x_{s+9}	Δc_0	$k_{s+1}x_{s+0} + k_{s+8}x_{s+2} + x_{s+0}x_{s+2} + x_{s+0}x_{s+3} + x_{s+0}x_{s+17} + x_{s+2}x_{s+10} + x_{s+2}x_{s+24}$
x_{s+10}	Δc_0	$k_{s+1} + x_{s+17} + x_{s+2}x_{s+9}$
x_{s+17}	Δc_0	$k_{s+8} + x_{s+10} + x_{s+24} + x_{s+0}x_{s+9}$
x_{s+24}	Δc_0	$k_{s+1} + x_{s+3} + x_{s+17} + x_{s+2}x_{s+9}$
x_{s+1}	Δc_1	$k_{s+2}x_{s+10} + x_{s+3}x_{s+10} + x_{s+4}x_{s+10} + x_{s+10}x_{s+18} + 1$
x_{s+3}	Δc_1	$k_{s+9}x_{s+10} + x_{s+1}x_{s+10} + x_{s+10}x_{s+11} + x_{s+10}x_{s+25}$
x_{s+4}	Δc_1	$k_{s+9} + x_{s+25} + x_{s+1}x_{s+10}$
x_{s+10}	Δc_1	$k_{s+2}x_{s+1} + k_{s+9}x_{s+3} + x_{s+1}x_{s+3} + x_{s+1}x_{s+4} + x_{s+1}x_{s+18} + x_{s+3}x_{s+11} + x_{s+3}x_{s+25}$
x_{s+11}	Δc_1	$k_{s+2} + x_{s+18} + x_{s+3}x_{s+10}$
x_{s+18}	Δc_1	$k_{s+9} + x_{s+11} + x_{s+25} + x_{s+1}x_{s+10}$
x_{s+25}	Δc_1	$k_{s+2} + x_{s+4} + x_{s+18} + x_{s+3}x_{s+10}$
$\vdots$	$\vdots$	$\vdots$

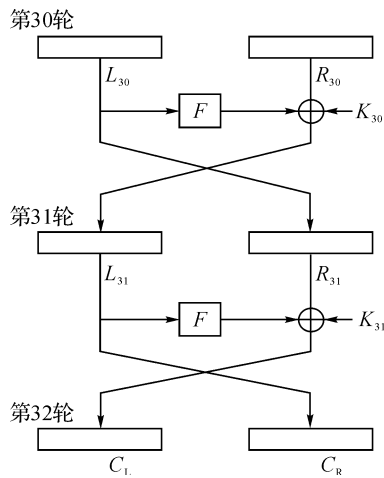


图 7 SIMON 最后 2 轮加密过程

Fig. 7 SIMON encryption process for last two rounds

观察表 5 发现,通过线性多项式攻击者可得到 $L_{30}(x_{s+0}, x_{s+1}, x_{s+2}, \dots, x_{s+15})$,分析第 32 轮加密过程,由 SIMON 轮函数可得

$$K_{31} = F(L_{31}) \oplus R_{31} \oplus C_L. \quad (9)$$

由于 $L_{31} = C_R, R_{31} = L_{30}$,利用式(9)易求得第 32 轮密钥 k_{31+i} .此外,利用表 5 中二次多项式还可求得 $R_{30} \oplus K_{30}$,猜测第 31 轮密钥 K_{30} ,可以得到 R_{30} ,此时第 30 轮中间状态全部获得.继续猜测第 30 轮和第 29 轮密钥,得到连续 4 轮密钥,进一步可恢复全轮密钥信息.将获得的密钥带入 1~30 轮加密过程(第 30 轮中间状态比特全部获得)验证猜测

表 5 SIMON32/64 第 30 轮多项式的筛选结果

Tab. 5 Selected equations of SIMON32/64 in round 30

故障注入位	密文差分	多项式
x_{s+1}	Δc_7	$k_{s+8} + x_{s+24} + x_{s+0}x_{s+9}$
	Δc_{14}	$k_{s+6} + x_{s+22} + x_{s+7}x_{s+14}$
	Δc_{16}	x_{s+8}
	Δc_{25}	x_{s+10}
x_{s+2}	Δc_8	$k_{s+9} + x_{s+25} + x_{s+1}x_{s+10}$
	Δc_{15}	$k_{s+7} + x_{s+23} + x_{s+8}x_{s+15}$
	Δc_{17}	x_{s+9}
	Δc_{26}	x_{s+11}
x_{s+5}	Δc_2	$k_{s+10} + x_{s+26} + x_{s+2}x_{s+11}$
	Δc_{11}	$k_{s+12} + x_{s+28} + x_{s+4}x_{s+13}$
	Δc_{20}	x_{s+12}
	Δc_{29}	x_{s+14}
x_{s+6}	Δc_3	$k_{s+11} + x_{s+27} + x_{s+3}x_{s+12}$
	Δc_{12}	$k_{s+13} + x_{s+29} + x_{s+5}x_{s+14}$
	Δc_{21}	x_{s+13}
	Δc_{30}	x_{s+15}
x_{s+9}	Δc_6	$k_{s+14} + x_{s+30} + x_{s+6}x_{s+15}$
	Δc_{15}	$k_{s+0} + x_{s+16} + x_{s+1}x_{s+8}$
	Δc_{17}	x_{s+2}
	Δc_{24}	x_{s+0}
x_{s+10}	Δc_0	$k_{s+1} + x_{s+17} + x_{s+2}x_{s+9}$
	Δc_7	$k_{s+15} + x_{s+31} + x_{s+0}x_{s+7}$
	Δc_{18}	x_{s+3}
	Δc_{25}	x_{s+1}
x_{s+13}	Δc_3	$k_{s+4} + x_{s+20} + x_{s+5}x_{s+12}$
	Δc_{10}	$k_{s+2} + x_{s+18} + x_{s+3}x_{s+10}$
	Δc_{21}	x_{s+6}
	Δc_{28}	x_{s+4}
x_{s+14}	Δc_4	$k_{s+5} + x_{s+21} + x_{s+6}x_{s+13}$
	Δc_{11}	$k_{s+3} + x_{s+19} + x_{s+4}x_{s+11}$
	Δc_{22}	x_{s+7}
	Δc_{29}	x_{s+5}

是否正确,计算复杂度为

$$\frac{30}{32} \times 2^{48} \approx 2^{47.91}.$$

故障注入后得到方程都是低次的,求解时间可忽略不计,因此故障立方攻击和密钥猜测总的计算复杂度约为 $2^{47.91}$.类似地,可求得对 SIMON48/72、SIMON64/96 攻击总计算复杂度分别为 $2^{47.92}$ 和 $2^{63.93}$.

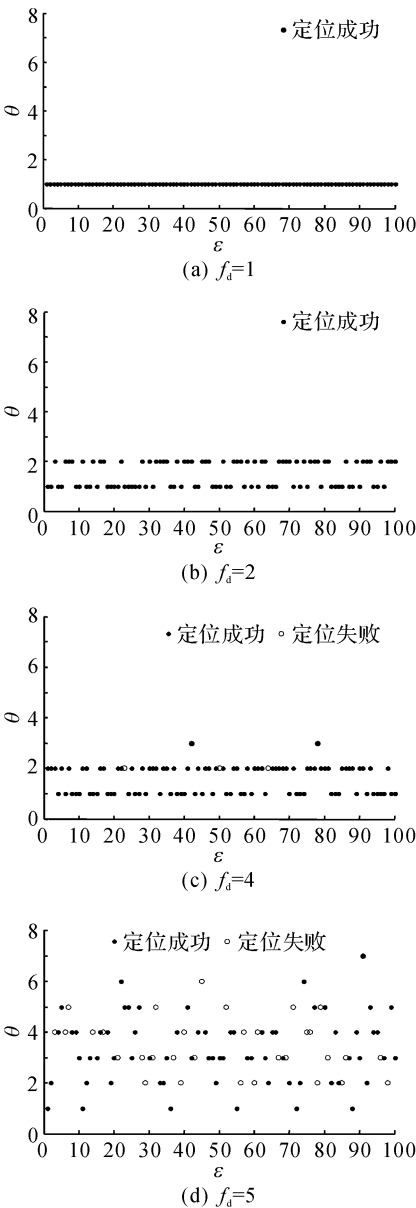


图 5 差分特征表故障定位仿真结果

Fig. 5 Simulation results of fault determination with difference-characteristics table

4.3 数据复杂度

用平均注入故障数来评估数据复杂度. 理论上, SIMON32/64 要覆盖表 5 中的故障注入位最少需要 8 次故障注入. 通过软件模拟故障随机注入过程, 对 SIMON32/64 进行 10 000 次攻击, 平均故障注入次数为 69 次, 多于文献[9]的仿真结果 25 次, 原因如下: 该文献假设能够对倒数第 2 轮左半部分进行故障注入, 假设条件较强. 同理, 可得到其余 2 种 SIMON 密码的故障注入情况: 对 SIMON48/72 进行 10 000 次攻击, 平均故障注入次数为 128 次; 对 SIMON64/96 进行 10 000 次攻击, 平均故障注入次数为 185 次.

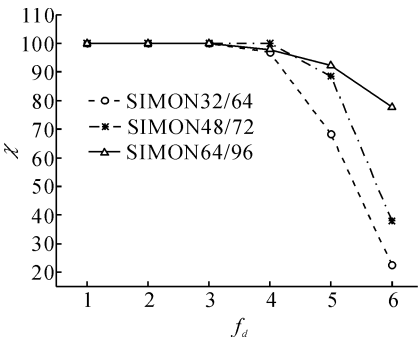


图 6 SIMON 系列密码差分表定位成功率

Fig. 6 Success rates for fault determination with difference-characteristics table of SIMON family ciphers

4.4 结果分析

本文对 SIMON32/64 密码的故障立方攻击结果与文献[8]进行的立方攻击研究成果对比如表 6 所示. 本文中 69 次故障注入相当于 $69 \times 2 \approx 2^{7.11}$ 个选择明文, 数据复杂度为 $2^{7.11}$. 相比于立方攻击和动态立方攻击, 故障立方攻击在数据复杂度和计算复杂度方面都更优.

表 6 SIMON32/64 立方攻击对比

Tab. 6 Comparison of cube attacks on SIMON32/64

攻击方法	攻击轮数	数据复杂度	计算复杂度	来源
动态立方攻击	14/32	2^{32}	$2^{51.09}$	文献[8]
立方攻击	17/32	2^{31}	$2^{51.49}$	文献[8]
故障立方攻击	32/32	$2^{7.11}$	$2^{47.91}$	本文

对比本文的故障立方攻击结果与文献[9]差分故障攻击结果. 以 SIMON32/64 为例, 本文通过对第 30 轮最少 8 次的故障注入, 利用提取的多项式可恢复第 32 轮 16 比特密钥, 与文献[9]中“倒数第 2 轮每一比特故障可恢复最后一轮 2bit 密钥”的结论一致. 相比于传统的差分故障攻击, 故障立方攻击存在下列优点:

- 1)故障定位方法有效性好,更易实现. 利用差分表进行故障定位可实现 SIMON32/64 倒数 4 轮任意故障注入位置定位, 成功率 $\geq 96\%$, 文献[9]仅能对倒数第 2 轮左半部分进行故障定位; 此外, 相比文献[9]单独对目标轮左半部分进行故障注入的模型, 本文采用的随机故障注入模型更易实现.
- 2)避免了繁琐的差分分析过程, 自动化程度高. 故障立方攻击将传统的手工分析转化为搜索线性或二次多项式的过程, 自动化程度高, 且对不同密码攻击过程是通用的.

5 结 语

本文给出一种故障立方攻击方法,对 SIMON 系列密码安全性进行评估研究. 结合代数攻击思想和立方攻击原理给出故障立方攻击所需推论,在此基础上分 3 步进行:确定故障注入轮、确定单比特故障注入位、利用立方攻击提取的低次多项式恢复部分轮密钥并结合密钥猜测攻击恢复全轮密钥. 结果表明:故障立方攻击在复杂度方面优于此前的立方攻击. 相比于差分故障攻击,本文的故障模型更加贴近实际,自动化程度较高. 此外,本文给出的方法通用性好,可为其他轻量级分组密码提供一定借鉴.

本文方法仍存在下列不足:1)由于不能直接求解密钥,搜索得到的一部分二次多项式并未得到充分利用;2)故障注入深度仅为倒数第 2 轮,能够覆盖到的轮密钥数量有限. 因此,如何充分利用二次多项式及增加故障注入深度是故障立方攻击下一步的研究方向.

参考文献 (References):

- [1] BEAULIEU R, SHORS D, SMITH J, et al. The SIMON and speck families of lightweight block ciphers [EB/OL]. (2013-06-19) [2016-11-30]. <http://eprint.iacr.org/2013/404.pdf>.
- [2] ALIZADEH J, BAGHERI N, GAURAVARAM P, et al. Linear cryptanalysis of round reduced SIMON [EB/OL]. (2014-10-16) [2016-11-30]. <http://eprint.iacr.org/2013/663.pdf>.
- [3] KÖLBL S, LEANDER G, TIESSEN T. Observations on the SIMON block cipher family [C] // **Proceedings of the 35th International Cryptology Conference**. Santa Barbara: Springer, 2015: 161-185.
- [4] ALIZADEH J, ALKHZAIMI H A, AREF M R, et al. Cryptanalysis of SIMON variants with connections [C] // **Proceedings of the 10th workshop on RFID Security**. Oxford: Springer, 2014: 90-107.
- [5] BIRYUKOV A, ROY A, VELICHKOV V. Differential analysis of block ciphers SIMON and SPECK [C] // **Proceedings of the 21st International Workshop on Fast Software Encryption**. London: Springer, 2015: 546-570.
- [6] ABED F, LIST E, LUCKS S, et al. Differential cryptanalysis of round-reduced simon and speck [C] // **Proceedings of the 21st International Workshop on Fast Software Encryption**. London: Springer, 2015: 525-545.
- [7] RADDUM H. Algebraic analysis of the simon block cipher family [C] // **Proceedings of the Fourth International Conference on Cryptology and Information Security in Latin America**. Guadalajara: Springer, 2015: 157-169.
- [8] RABBANINEJAD R, AHMADIAN Z, SALMASIZADEH M, et al. Cube and dynamic cube attacks on SI-

- MON32/64 [C] // **Proceedings of the 11th International ISC Conference on Information Security and Cryptology**. Piscataway: IEEE, 2014: 98-103.
- [9] TUPSAMUDRE H, BISHT S, MUKHOPADHYAY D. Differential fault analysis on the families of SIMON and SPECK ciphers [EB/OL]. (2014-05-30) [2016-11-30]. <http://eprint.iacr.org/2014/267.pdf>.
- [10] YANG L, WANG M Q, QIAO S Y. Side channel cube attack on PRESENT [C] // **Proceedings of the International Conference on Cryptology and Network Security**. Kanazawa: Springer, 2009: 379-391.
- [11] ABDUL-LATIP S F, REYHANITABAR M R, SUSILO W, et al. On the security of NOKEON against side channel cube attacks [C] // **Proceedings of the 6th Information Security Practice and Experience Conference**. Seoul: Springer, 2010: 45-55.
- [12] 赵新杰,郭世泽,王韬,等. EPCBC 密码旁路立方体攻击 [J]. 成都信息工程学院学报, 2012, 27(6): 525-530.
ZHAO Xin-jie, GUO Shi-ze, WANG Tao, et al. Side-channel cube attacks on EPCBC [J]. **Journal of Chengdu University of Information Technology**, 2012, 27(6): 525-530.
- [13] LI Z Q, ZHANG B, YAO Y, et al. Cube cryptanalysis of LBlock with noisy leakage [C] // **Proceedings of the 15th Annual International Conference on Information Security and Cryptology**. Seoul: Springer, 2013: 141-155.
- [14] 曾文. Trivium 算法的 Fault Cube 攻击与可滑动对研究 [D]. 郑州:信息工程大学, 2011.
ZENG Wen. The fault cube attack and slid pairs research on trivium [D]. Zhengzhou: Information Engineering University, 2011.
- [15] ABDUL-LATIP S F, REYHANITABAR M R, SUSILO W, et al. Fault analysis of the KATAN family of block ciphers [C] // **Proceedings of the 8th Information Security Practice and Experience Conference**. Hangzhou: Springer, 2012: 319-336.
- [16] DINUR I, SHAMIR A. Cube attacks on tweakable black box polynomials [C] // **Proceedings of the 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques**. Cologne: Springer, 2009: 278-299.
- [17] 郭世泽,王韬,赵新杰. 密码旁路分析原理与方法 [M]. 北京:科学出版社, 2014: 248-277.
- [18] BONEH D, DEMILLO R A, LIPTON R J. On the Importance of checking cryptographic protocols for faults [C] // **Proceedings of the 15th Annual EURO-CRYPT Conference on the Theory and Applications of Cryptologic Techniques**. Konstanz: Springer, 1997: 37-51.
- [19] BIHAM E, SHAMIR A. Differential fault analysis of secret key cryptosystems [C] // **Proceedings of the 17th Annual International Cryptology Conference**. Santa Barbara, US: Springer, 1997: 513-525.
- [20] 马克裘依,腾斯托尔. 密码故障分析与防护 [M]. 赵新杰,郭世泽,张帆,等,译. 北京:科学出版社, 2015: 240-245.